

Chapter 7. Child Processes

Running a Node instance in a single process works fine for a small application. As the application gets more complex and serves more users, a single process is not going to be enough to handle the increasing workload. No matter how powerful your server may be, a single thread can support only a limited load.

The fact that your Node code runs in a single thread does not mean that you can't take advantage of multiple processes and, of course, multiple machines as well.

Using multiple processes is the best way to scale a Node application. Node is designed for building distributed applications with many nodes. This is why it's named Node! Scalability is baked into the platform, and it's not something you start thinking about later in the lifetime of an application.

Introducing Child Processes

We can easily create a child process in a main Node process using the `node:child_process` built-in module. These child processes can communicate with each other with an event-driven messaging system because process objects implement the `EventEmitter` structure under the hood.

The `node:child_process` module enables us to efficiently run OS commands (like `find`, `grep`, etc.) from within a Node process, safely control the arguments passed to these commands, and do whatever we want with the commands' output. This gives a Node process a lot more power over what it can do.

In addition, the `node:child_process` module supports both buffers and streams to process the input and output of child processes. With streams,

we can combine the power of multiple commands and run them together even if the task involves working with large amounts of data. We can simply pipe the output of one command as the input to another.

The `node:child_process` module offers four different ways to create a child process: `spawn()`, `fork()`, `exec()`, and `execFile()`. We're going to see the differences between these four functions and when to use each.

The spawn Function

The `spawn` function launches a command in a new process, and we can use it to pass that command any arguments. For example, here's code to spawn a new process that will execute the `pwd` Linux command:

```
import { spawn } from 'node:child_process';

const child = spawn('pwd');
```

We import the `spawn` function and execute it with the OS command as the first argument.

The result of executing the `spawn` function (the `child` object in this example) is a `ChildProcess` instance, which implements the `EventEmitter` API. This means we can register handlers for events on this `child` object directly. For example, we can do something when the child process exits by registering a handler for the `exit` event:

```
child.on('exit', function(code, signal) {
  console.log(
    `Child process exited. Code: ${code} - Signal: ${signal}`
  );
});
```

The handler function for an `exit` event gets the exit `code` and the `signal` that was used to terminate the child process. The `signal` variable is `null` when the child process exits normally.

Another important event to handle is the `error` event, which is emitted if the process could not be spawned (or terminated), or if it had any issues while executing. It's a good practice to always have a handler for this event:

```
child.on('error', (err) => {  
  console.error(`Child process encountered an error: ${err.message}`);  
});
```

The other events that we can register handlers for with the `ChildProcess` instances are `message`, `spawn`, `disconnect`, and `close`:

The `message` event

Emitted when the child process uses the `process.send()` function to send messages. This is how parent/child processes can communicate with each other. We'll see an example of that shortly.

The `spawn` event

Emitted once the child process has spawned successfully.

The `disconnect` event

Emitted when the parent process manually calls the `child.disconnect` method.

The `close` event

Emitted when the I/O streams of a child process are terminated. Every child process gets the three standard `stdio` streams, which we can access using `child.stdin`, `child.stdout`, and `child.stderr`. When these streams get closed, the child process that was using them emits the `close` event.

TIP

This `close` event is different from the `exit` event because multiple child processes might share the same `stdio` streams, so one child process exiting does not mean that the streams got closed.

Since all streams are event emitters, we can listen to different events on the `stdio` streams that are attached to every child process. Unlike in a normal process, though, in a child process the `stdout` / `stderr` streams are readable streams while the `stdin` stream is a writable one. This is basically the inverse of those types found in a main process.

The events we can use for the `stdio` streams are the standard ones. Most importantly, on the readable streams we can listen to the `data` event, which will have the output of the command or any error encountered while executing the command:

```
child.stdout.on('data', data => {
  console.log(`child stdout:\n${data}`);
});

child.stderr.on('data', data => {
  console.error(`child stderr:\n${data}`);
});
```

The two handlers in this example will log both cases to the main process `stdout` and `stderr`. If we execute the code as it stands, the output of the `pwd` command gets printed, and the child process exits with code `0`, which means no error occurred.

We can pass arguments to the command that's executed by the `spawn` function using the second argument of the `spawn` function, which is an array of all the arguments to be passed to the command. For example, to execute the `find` command on the current directory with a `-type f` argument (to list files only), we can do:

```
const child = spawn('find', ['.', '-type', 'f']);
```

If an error occurs during the execution of the command (for example, if we give the preceding `find` command an invalid destination), the `data` event handler on the `child.stderr` stream will be triggered, and the `exit` event handler on that instance will report an exit code of `1`, which

signifies that an error has occurred. The error values actually depend on the host OS and the type of error.

A child process `stdin` is a writable stream. We can use it to send a command some input. Just like any writable stream, the easiest way to consume it is using the `pipe` function. We simply pipe a readable stream into a writable stream. Since the main process `stdin` is a readable stream, we can pipe that into a child process `stdin` stream. Here's an example:

```
import { spawn } from 'node:child_process';

const child = spawn(`wc`);

process.stdin.pipe(child.stdin);

child.stdout.on(`data`, data => {
  console.log(`child stdout:\n${data}`);
});
```

In this example, the child process invokes the `wc` command, which counts lines, words, and characters in Linux. We then pipe the main process `stdin` (which is a readable stream) into the child process `stdin` (which is a writable stream). The result of this combination is that we get a standard input mode where we can type something, and when we hit Ctrl + D, what we type will be used as the input of the `wc` command.

We can also pipe the standard I/O of multiple processes on each other, just like we can do with Linux commands. For example, we can pipe the `stdout` of the `find` command to the `stdin` of the `wc` command to count all the files in the current directory:

```
import { spawn } from 'node:child_process';

const find = spawn('find', ['.', '-type', 'f']);
const wc = spawn('wc', ['-l']);

find.stdout.pipe(wc.stdin);
```

```
wc.stdout.on('data', data => {  
  console.log(`Number of files ${data}`);  
});
```

I added the `-l` argument to the `wc` command to make it count only the lines. When executed, the preceding code will output a count of all files in all directories under the current one.

Shell Syntax and the exec Function

By default, the `spawn` function does not create a *shell* to execute the command we pass to it. This makes it slightly more efficient than the `exec` function, which does create a shell. The `exec` function has one other major difference. It buffers the command's generated output and passes the whole output value to a callback function (instead of using streams, which is what `spawn` does).

Since the `exec` function uses a shell to execute the command, we can use any *shell syntax* directly with it. Shell syntax is the set of rules that guide how commands are structured and interpreted in a shell environment. For example, commands can have options and arguments.

There are also some special characters that we can use with commands. For example, the pipe symbol can be used to connect the output of one command to the input of another, the `*` character can be used in some commands to match files based on patterns, and the `>` character can be used to redirect the output of a command. You can even use loops and conditions and many other advanced features in shell commands.

Here's the previous `find/wc` example implemented with an `exec` function:

```
import { exec } from 'node:child_process';  
  
exec('find . -type f | wc -l', (err, stdout, stderr) => {  
  if (err) {  
    console.error(`exec error: ${err}`);  
  }  
});
```

```
    return;  
  }  
  
  console.log(`Number of files ${stdout}`);  
});
```

Note how I used the shell native pipe feature through the pipe symbol (`|`) in this example.

WARNING

Be careful when using the shell syntax in Node as it comes with a security risk, especially if you're executing any input that's provided externally. A user can simply initiate a command injection attack using shell syntax characters like `;` (for example, `command + ' ; rm -rf ~'`). Strategies to mitigate that include sanitizing and validating input, making a list of acceptable commands, and restricting access control.

The `exec` function buffers the output and passes it to the callback function (the second argument to `exec`) as the `stdout` argument there. This `stdout` argument is the command's output that we want to print out.

The `exec` function is a good choice if you need to use the shell syntax and the size of the data expected from the command is small, since it buffers the whole data in memory before returning it.

The `spawn` function, with its stream objects, is a much better choice when the size of the output expected from the command is large. Using streams is compatible with the standard I/O objects that are often used with child processes. We can even make a spawned child process inherit the standard I/O objects of its parents if we want to. More importantly, we can make the `spawn` function use the shell syntax as well. Here's the same `find | wc` command implemented with the `spawn` function:

```
const child = spawn('find . -type f | wc -l', {  
  stdio: 'inherit',
```

```
    shell: true
  });
```

With the `stdio: 'inherit'` option, when we execute the code, the child process inherits the main process `stdin`, `stdout`, and `stderr`. This causes the child process data events handlers to be triggered on the main `process.stdout` stream, making the script output the result right away.

The `shell: true` option enables us to use the shell syntax in the executed command, just like we did with `exec`. But with this code, we still get the advantage of the streaming of data that the `spawn` function gives us. This is really the best of both worlds.

There are a few other good options we can use in the last argument to the `node:child_process` functions besides `shell` and `stdio`. For example, we can use the `cwd` option to change the working directory of the script. Here's the same count-all-files example done with a `spawn` function using a shell and with a working directory set to my *Downloads* folder:

```
const child = spawn('find . -type f | wc -l', {
  stdio: 'inherit',
  shell: true,
  cwd: '/Users/samer/Downloads'
});
```

Another option we can use is `env` to specify the environment variables that will be visible to the new child process. The default for this option is `process.env`, which gives any command access to the current process environment. If we want to override that behavior, we can simply pass an empty object as the `env` option or new values there to be considered as the only environment variables:

```
const child = spawn('echo $ANSWER', {
  stdio: 'inherit',
  shell: true,
  env: { ANSWER: 42 }
});
```


Since we specified an `env` object, the `echo` command will not have access to the parent process's environment variables. For example, it cannot access `$HOME`, but it can access `$ANSWER` because it was defined in the `env` property.

One last important child process option to explain here is the `detached` option, which makes the child process run independently of its parent process.

Assume we have a file, *timer.js*, that keeps the event loop busy:

```
setTimeout(() => {  
  // Keep the event loop busy  
}, 20_000);
```

We can execute *timer.js* in the background using the `detached` option:

```
import { spawn } from 'node:child_process';  
  
const child = spawn('node', ['timer.js'], {  
  detached: true,  
  stdio: 'ignore'  
});  
  
child.unref();
```

The exact behavior of detached child processes depends on the OS. On Windows, they will have their own console window while on Linux they will be made the leaders of new process groups and sessions.

If the `unref` function is called on the detached process, the parent process can exit independently of the child. This can be useful if the child is executing a long-running process, but to keep it running in the background, the child's `stdio` configurations also have to be independent of the parent.

This example will run the *timer.js* script in the background by detaching and also ignoring its parent `stdio` file descriptors so that the parent can

terminate while the child keeps running in the background.

The execFile Function

The `execFile` function behaves exactly like the `exec` function, but it does not use a shell, which makes it a bit more efficient and secure (because it has no shell syntax). We can use it to execute an external program or a script. Here's an example:

```
import { execFile } from 'node:child_process';

execFile('ruby', ['-e', 'puts rand(1..100)'], (error, stdout, stderr) => {
  if (error) {
    console.error(`Error: ${error}`);
    return;
  }

  console.log(`Output: ${stdout}`);
});
```

This example uses `execFile` to execute the `ruby` program, setting the arguments to execute a one-liner that'll print a random number between 1 and 100.

WARNING

On Windows, some files cannot be executed on their own, like `.bat` or `.cmd` files. Those files cannot be executed with `execFile`. Either `exec` or `spawn` with `shell` set to true is required to execute them.

SYNCHRONOUS CHILD PROCESS FUNCTIONS

The functions `spawn`, `exec`, and `execFile` from the `node:child_process` module also have synchronous blocking versions that will wait until the child process exits:

```
import {
  spawnSync,
  execSync,
  execFileSync
} from 'node:child_process';
```

Those synchronous versions are potentially useful when trying to simplify scripting tasks or any startup processing tasks, but they should be avoided otherwise.

The fork Function

The `fork` function is a variation of the `spawn` function for spawning node processes. The biggest difference between `spawn` and `fork` is that a channel for interprocess communication (IPC) is established to the child process when using `fork`, so we can use the `send` function on the forked `process` along with the `process` object itself to exchange messages between the parent and forked processes. We do this through the `EventEmitter` module interface. Here's an example.

In the parent file, *parent.js*, we have the following:

```
import { fork } from 'node:child_process';

const forked = fork('child.js');

forked.on('message', msg => {
  console.log('Message from child', msg);
});

forked.send({ hello: 'world' });
```

In the child file, *child.js*, we have the following:

```
process.on('message', msg => {
  console.log('Message from parent:', msg);
});

let counter = 0;

setInterval(() => {
  process.send({ counter: counter++ });
}, 1_000);
```

In the parent file, we fork *child.js* (which will execute the file with the `node` command), and then we listen for the `message` event. The `message` event will be emitted whenever the child uses `process.send`, which we're doing every second.

To pass down messages from the parent to the child, we can execute the `send` function on the forked object itself. Then, in the child script, we can listen to the `message` event on the `process` object.

When executing the preceding *parent.js* file, it'll first send down the `{ hello: 'world' }` object to be printed by the forked child process. Then the forked child process will send an incremented counter value every second to be printed by the parent process.

Let's look at a more practical example of using the `fork` function. Let's say we have a web server that handles two endpoints. One of these endpoints (`/compute`) is computationally expensive and will take a few seconds to complete. We can use a long `for` loop to simulate that:

```
import { createServer } from 'node:http';

const longComputation = () => {
  let sum = 0;
  for (let i = 0; i < 1e9; i++) {
    sum += i;
  }
  return sum;
};
```

```

};

const server = createServer();

server.on('request', (req, res) => {
  if (req.url === '/compute') {
    const sum = longComputation();
    return res.end(`Sum is ${sum}`);
  } else {
    res.end('Ok');
  }
});

server.listen(3000, () => {
  console.log('Server is running...');
});

```

This program has a big problem; when the `/compute` endpoint is requested, the server will not be able to handle any other requests because the main thread is busy with the long `for` loop operation.

There are a few ways we can solve this problem depending on the nature of the long operation, but one solution that works for all cases is to just move the computational operation into another process using the `fork` function.

We first move the whole `longComputation` function into its own file and make it invoke that function when instructed via a `message` event from the main process. In a new *compute.js* file:

```

const longComputation = () => {
  let sum = 0;
  for (let i = 0; i < 1e9; i++) {
    sum += i;
  }
  return sum;
};

process.on('message', msg => {
  const sum = longComputation();

```

```
    process.send(sum);  
  });
```

Now, instead of doing the long operation in the main thread, we can fork the `compute.js` file and use the messages interface to communicate messages between the server and the forked process:

```
import { createServer } from 'node:http';  
import { fork } from 'node:child_process';  
  
const server = createServer();  
  
server.on('request', (req, res) => {  
  if (req.url === '/compute') {  
    const compute = fork('compute.js');  
    compute.send('start');  
    compute.on('message', sum => {  
      res.end(`Sum is ${sum}`);  
    });  
  } else {  
    res.end('Ok');  
  }  
});  
  
server.listen(3000, () => {  
  console.log('Server is running...');  
});
```

When a request to `/compute` happens now with this code, we simply send a message to the forked process to start executing the long operation. The main thread will not be blocked.

Once the forked process is done with that long operation, it can send its result back to the parent process using `process.send`.

In the parent process, we listen to the `message` event on the forked child process itself. When we get that event, we'll have a `sum` value ready for us to send to the requesting user over HTTP.

This method is limited by the number of processes we can fork, but when we execute it and request the long computation endpoint over HTTP, the main server is not blocked at all and can take further requests.

Node's cluster module, which is the topic of [Chapter 9](#), is based on this idea of child-process forking and load balancing the requests among the many forks that we can create on any system.

Summary

We can create child processes in Node to scale applications beyond a single thread and access external functionalities from the OS and other programs.

Node child processes enable us to execute external shell commands, programs, and scripts. We can also establish communication between child processes and their parent process when we need to.

The `child_process` module in Node has various flexible asynchronous functions like `spawn`, `exec`, and `execFile` and matching synchronous versions that will wait until the child process exits. The `spawn` function can be used to stream the output of execution, making it suitable for handling large outputs. Shell syntax can be used in `spawn` and `exec`.

The `fork` function can be used to delegate running a Node script to a different process and free the main process to do more.

Child process objects are event emitters. They emit events like `exit`, `error`, and `message`. The `message` event is how a child process can communicate with its parent process.

In the next chapter, we'll learn about the Node's built-in modules for writing tests and assertions.